Proposal: Field-Level Encryption of Personal Data

Backend (Express + MongoDB Atlas) | For approval by Tech Lead

1. Summary

We store personal and financial data in MongoDB Atlas as plain text: phone numbers, addresses, Aadhaar/PAN/licence numbers, bank accounts, chat and support text. Anyone who gets read access to the database or a backup (leaked connection string, misconfigured access, compromised Atlas account, exported backup) can read all of it.

Proposal: encrypt sensitive fields in the backend before they are saved to Atlas, and decrypt them in the backend when they are read, so API responses and the mobile apps/admin panel keep working unchanged. The database then only ever holds ciphertext for those fields.

2. How it works

- **Write:** a Mongoose plugin encrypts the listed fields on save/update, just before they reach Atlas.
- **Read:** the same plugin decrypts them on find/findOne (including `.lean()` queries), so controllers and `res.json` see normal values. No API contract change; no change in the apps or admin panel.
- **Cipher:** AES-256-GCM (authenticated encryption) with a random IV per value. The same cipher is already used in `backend/src/utils/crypto.ts` for DigiLocker tokens, so this extends proven code.
- **Format:** each stored value carries a key-version prefix (e.g. `v1.<iv>.<tag>.<data>`), which allows key rotation later.
- **Per-model opt-in:** each model lists its sensitive paths in one line; nothing else in the codebase has to change.

3. What gets encrypted

Collection	Encrypted fields	Notes
Driver (Phase 1)	Aadhaar no., PAN no., licence no., bank account no., IFSC, KYC image URLs (PAN, licence, selfie)	Most sensitive, rarely queried by value
User (Phase 2)	Saved addresses (address line, landmark, phones, receiver name), booking contact number, profile picture URL	Address coordinates stay plain
Vendor (Phase 2)	Owner PAN, GSTIN, bank account, IFSC, DigiLocker identity data, cheque/KYC file names	
Orders, Chat, Support (Phase 3)	Package-delivery contacts, chat text, support ticket text	Needs review of search/aggregation use

Deliberately not encrypted: geo fields (`location`, `currentLocation`, `stops.location`) because dispatch and nearby search use MongoDB 2dsphere queries that cannot run on ciphertext; statuses, timestamps, prices; passwords (already bcrypt hashes). **Phone and email** are excluded from the first phases because login/OTP look users up by them (see Risks).

4. Rollout plan

Step	Work	Outcome
1	Generalise crypto helper (new `DATA_ENCRYPTION_KEY`, key version prefix)	Reusable encrypt/decrypt

Step	Work	Outcome
2	Build Mongoose plugin + unit tests	One-line opt-in per model
3	Apply to Driver model; test onboarding, KYC, payouts, admin driver views	Phase 1 live; old plain values still readable
4	Migration script (`backend/src/scripts/`): encrypt existing data in batches, idempotent, after a backup	Existing records encrypted
5	Extend to User, Vendor, then Orders/Chat/Support, one collection at a time	Full coverage

Deploying the plugin before migrating is safe: values without the prefix are treated as plain text and returned as they are, so there is no downtime and each phase can be rolled back by removing the plugin line before migration.

5. Risks and mitigations

Risk	Mitigation
Queries filtering by an encrypted value stop matching (random IV)	Do not encrypt searched fields in early phases. If phone/email are needed later, add a keyed-hash (HMAC) lookup field and update auth/OTP/admin search.
`.aggregate()` bypasses Mongoose hooks (about 160 uses of .lean()/.aggregate() across 41 files)	Audit every aggregation touching encrypted fields before each phase; decrypt in code where needed.
Lost or leaked key	Key stored in a secrets manager (not in the repo); backed up separately. Losing it means encrypted data is unrecoverable, so backup is mandatory before rollout.
Backend compromise still exposes data (backend holds the key)	Scope is protection against database/backup/Atlas-side exposure. Combine with Atlas encryption at rest and least-privilege access.
Plain values leaked via logs	Review console logging of user/driver objects; do not log decrypted fields.
Migration errors	Full Atlas backup first; idempotent batches; run on staging copy; verify counts before production.

6. Alternatives considered

Option	Why not chosen
Atlas encryption at rest only	Does not protect against anyone who can query the database. We will enable it in addition.
MongoDB CSFLE / Queryable Encryption	Stronger and supports equality search, but needs native crypto libraries, schema maps, a KMS, and Mongoose compatibility testing. Considered a later upgrade if required.
End-to-end encryption from the apps	Not workable: the server must read addresses, phones and locations for dispatch and support.

7. Decisions requested

- Approve backend-side field encryption (AES-256-GCM via Mongoose plugin) as the approach.
- Approve Phase 1 scope: Driver KYC and bank fields, with the migration script reviewed before it runs on production data.
- Confirm where the encryption key will live (secrets manager vs environment variable) and who owns key backup.

- Confirm whether phone and email should be encrypted later (needs a hash lookup field and auth/OTP changes), or stay protected by access control only.